

Complete Guide

Setup, Usage, Architecture & Diagrams

Project: AI_ML_GENAI

Getting Started

This guide walks you through setting up the AI / ML / GenAI workspace from scratch.

Prerequisites

Requirement Details

Step 1: Install uv

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Restart your terminal or add uv to PATH:

```
$env:Path = "$env:USERPROFILE\.local\bin;$env:Path"
```

Step 2: Clone / open project

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Step 3: Install dependencies

```
uv sync --extra dev
```

This creates .venv/ and installs all core packages.

Optional - local GGUF models:

```
uv sync --extra local-llm
```

Requires C++ build tools on Windows. Skip this if you use an API LLM.

Step 4: Configure environment

```
copy .env.example .env
```

Edit .env for your setup. See Configuration Reference.

Minimum for API usage:

```
USE_API_LLM=true
```

Step 5: Add documents (for RAG)

Place files in data/raw/:

* PDF (.pdf)

* Tex

Step 6: Build vector index

```
uv run python ingest.py
```

Output example: Indexed 142 chunks into vector_db

Step 7: Run the application

```
uv run streamlit run app.py
```

Open the URL shown in the terminal (usually <http://localhost:8501>).

Step 8: Verify installation

```
uv run pytest
```

All tests should pass.

Run, activate, and add dependencies

For daily use - activating `.venv`, running all commands, and adding packages with `uv` - see Run, Activate & Dependencies.

Quick reference:

```
# Activate (optional - uv run works without activation)
```

.\.venv

What to do next

Goal Read

How to Run, Activate, and Manage Dependencies

This guide covers daily development workflow: activating the environment, running the project, and adding new packages with uv.

Table of contents

1. Activate the virtual environment

2. How

Activate the virtual environment

This project uses uv and a virtual environment at .venv/ (created automatically on first uv sync).

You have two options - Option A is recommended because you do not need to activate manually.

Option A: Use uv run (recommended - no activation needed)

uv run automatically uses the project .venv for any command:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

No activate step required.

Option B: Activate .venv manually

Use this if you prefer a traditional workflow or run many commands in the same shell session.

Windows PowerShell:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

.\.venv

Windows Command Prompt (cmd):

```
cd C:\MY_SPACE\AI_ML_GENAI
```

.venv\

Linux / macOS:

```
cd /path/to/AI_ML_GENAI
```

source

Verify activation - your prompt should show (.venv) and Python should point to the venv:

```
python --version
```

where

Deactivate when done:

```
deactivate
```

If .venv does not exist yet

Create it and install dependencies first:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

How to run the project

Always start from the project root:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

First-time / full setup run

```
# 1. Install dependencies (creates .venv)
```

Open the browser URL from the terminal (usually http://localhost:8501).

Run commands reference

What to run	Command	Notes
-------------	---------	-------

Run with activated venv (Option B)

If you activated .venv manually, drop the uv run prefix:

```
.\.venv\Scripts\Activate.ps1
```

Optional: local GGUF model support

```
uv sync --extra local-llm
```

Then place a .gguf file in models/ and set LLM_MODEL_PATH in .env.

Stop the application

* Streamlit / Jupyter: Press Ctrl + C in the terminal

* MC

Add new dependencies

Dependencies are managed in pyproject.toml using uv (not pip install directly).

Add a runtime dependency

```
cd C:\MY_SPACE\AI_ML_GENAI
```

uv ad

Examples:

```
uv add fastapi          # Add REST API framework
```

uv ad

This updates pyproject.toml and uv.lock, and installs into .venv.

Add a development-only dependency

```
uv add --dev ruff        # Linter
```

uv ad

Dev dependencies go under [project.optional-dependencies] or uv's dev group.

Add an optional dependency group

For optional features (like local-llm), edit pyproject.toml or use:

```
uv add --optional local-llm llama-cpp-python
```

Then install with:

```
uv sync --extra local-llm
```

Install all dependencies after pulling changes

If someone else added packages or you cloned the repo:

```
uv sync --extra dev
```

Install with optional extras:

```
uv sync --extra dev --extra local-llm
```

Remove a dependency

```
uv remove <package-name>
```

Upgrade a dependency

```
uv add <package-name> --upgrade
```

List installed packages

```
uv pip list
```

Or with activated venv:

```
pip list
```

Important: do not use plain pip install for project deps

Avoid Use instead

Using uv add keeps pyproject.toml and uv.lock in sync for reproducible installs.

Daily workflow cheat sheet

```
# Open project
```

```
cd C:\
```

Related docs

* Getting Started - first-time setup

* Us

User Manual

Complete guide for using the AI / ML / GenAI workspace.

Table of contents

1. Application overview
2. Mo

Application overview

The main entry point is app.py, a Streamlit web UI with four modes selectable from the sidebar:

Mode	Module	Best for

Application modes flow

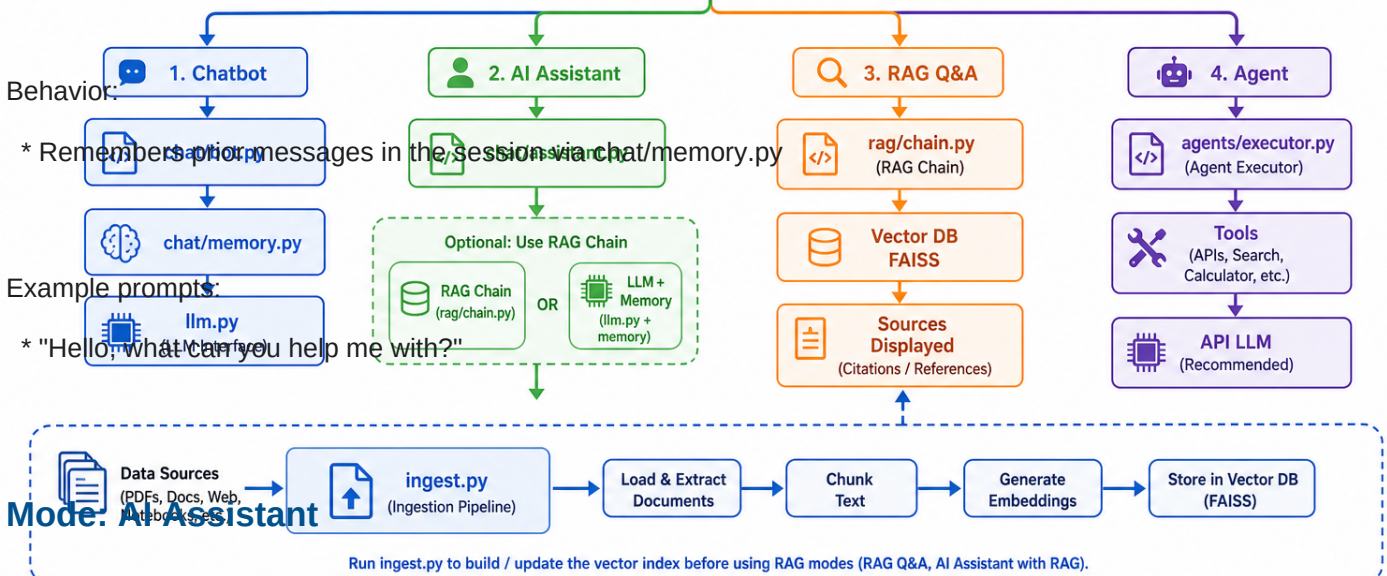
Application Modes - app.py Streamlit UI

Mode: Chatbot

Purpose: Simple conversational chat with conversation history.

How to use:

1. Run `uv run streamlit run app.py`



Mode: AI Assistant

Purpose: General-purpose assistant that can optionally ground answers in your documents.

How to use:

1. Select AI Assistant in the sidebar

Behavior:

- * With RAG enabled: searches vector_db/ and answers from retrieved context

When to use:

- * General help with optional document grounding

Mode: RAG Q&A

Purpose: Answer questions only from documents in data/raw/.

Prerequisites:

1. Documents in data/raw/

How to use:

1. Select RAG Q&A in the sidebar

Behavior:

- * Retrieves top-K relevant chunks (TOP_K in .env, default 4)

Example workflow:

```
# 1. Add report.pdf to data/raw/
```

Mode: Agent

Purpose: Run an AI agent that can call tools to complete tasks.

Prerequisites:

- * Set USE_API_LLM=true and OPENAI_API_KEY in .env (recommended)

How to use:

1. Select Agent in the sidebar

Available tools (agents/tools.py):

Tool	Description

Multi-step workflows (agents/workflows.py):

```
from agents.workflows import research_and_summarize
```

Document ingestion

Script: ingest.py

Pipeline:

1. Load documents from data/raw/ (rag/loader.py)

```
uv run python ingest.py
```

Re-run after:

- * Adding new documents

Supported formats: PDF, TXT, Markdown

MCP server

Purpose: Expose project tools to external clients (Cursor, Claude Desktop, etc.) via Model Context Protocol.

Start server:

```
uv run python mcp_server/server.py
```

Built-in tools:

Tool	Description
------	-------------

Cursor configuration - reference `mcp_server/config.json`:

 $\{$

Add custom tools: Register new functions in `mcp_server/server.py` or add helpers in `mcp_server/tools/`.

Notebooks

Location: notebooks/experiments.ipynb

Use for:

- * Testing chunk sizes before full ingestion

```
uv run jupyter lab notebooks/
```

Models

See Models Guide for:

- * OpenAI API vs local GGUF

* Wh

Command reference

Full run/activate/dependency guide: Run, Activate & Dependencies

Activate virtual environment (optional)

```
..\env\Scripts\Activate.ps1 # PowerShell
```

deact:

Or use uv run before any command - no activation needed.

Run commands

Command	Description
---------	-------------

Setup & dependencies

Command	Description
---------	-------------

Architecture

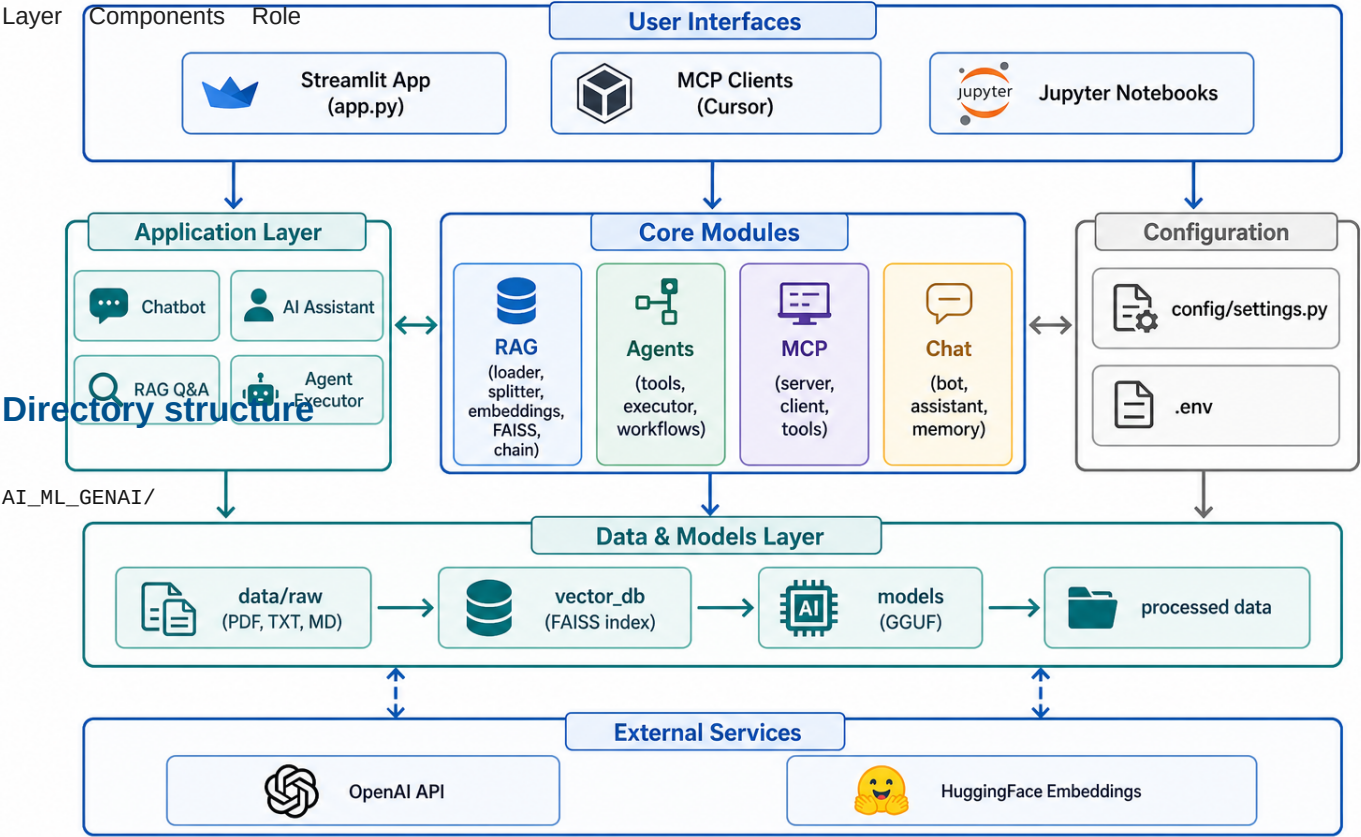
System design and module responsibilities for the AI / ML / GenAI workspace.

High-level architecture

System architecture overview

The project follows a layered modular architecture:

AI ML GenAI - System Architecture



Module: config/

File: config/settings.py

Central configuration loaded from environment variables via python-dotenv.

Setting Purpose

All modules import from here - no hardcoded paths in business logic.

Module: rag/

Retrieval-Augmented Generation pipeline.

File Responsibility

Design choice: RAG is isolated so it can be used by app.py, chat/assistant.py, and future APIs without duplication.

Module: agents/

Tool-calling AI agents.

File Responsibility

Design choice: Agents use LangChain's create_tool_calling_agent pattern. Tools are registered in one place for easy extension.

Module: mcp_server/

Model Context Protocol integration.

File Responsibility

Design choice: MCP runs as a separate process, enabling integration with Cursor and other MCP-compatible clients without modifying the main app.

Module: chat/

Conversational interfaces.

File Responsibility

Design choice: Chat and RAG are composable - AIAssistant optionally loads the RAG chain when vector_db/ exists.

Data flow summary

Flow Path

See Flow Diagrams for detailed step-by-step flows.

Technology stack

Component Technology

Extension points

Want to add... Where to start

Flow Diagrams

Step-by-step flows for each major capability. Each section includes a PNG diagram and a Mermaid flowchart (for editors that support it).

1. Overall application flow

flowchart TD

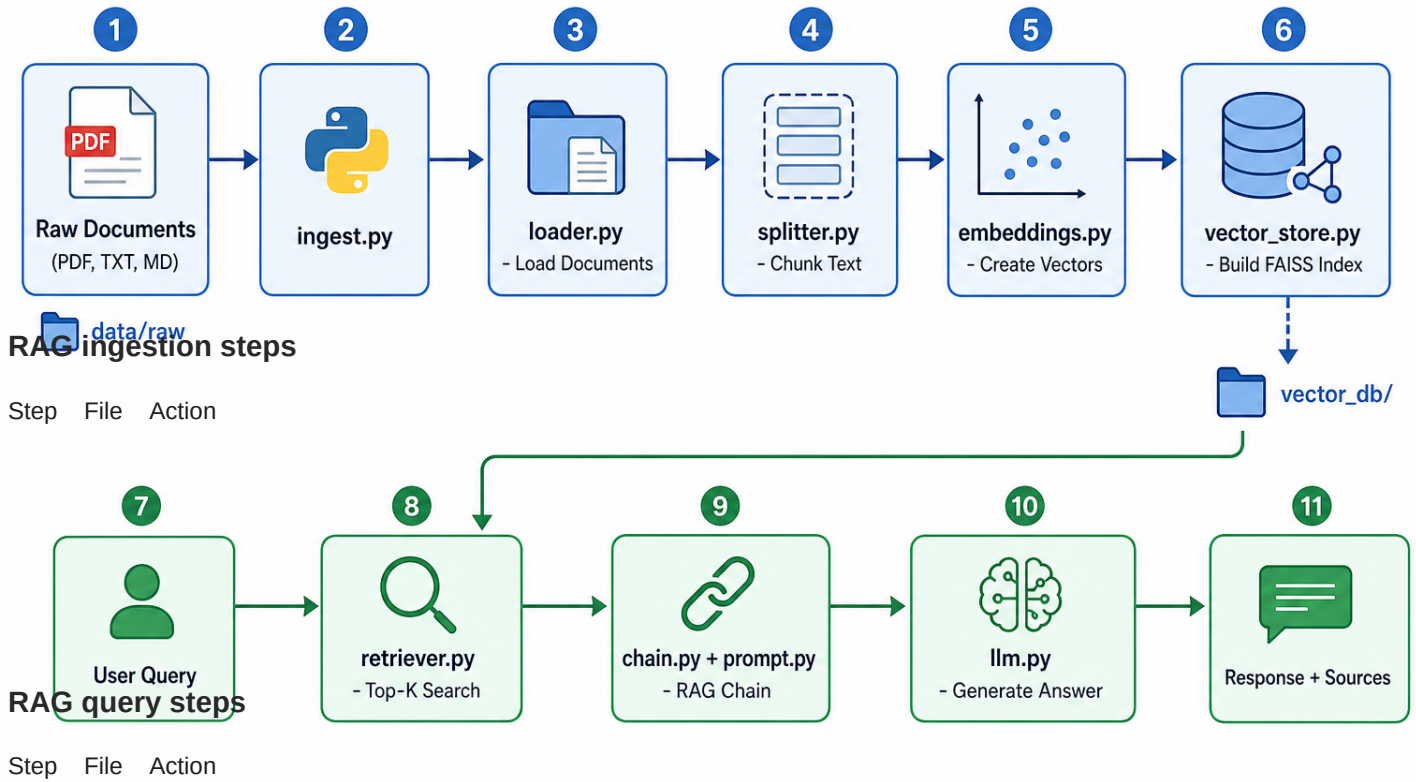
A

2. RAG pipeline flow

RAG pipeline flow

flowchart LR

RAG Pipeline Flow



3. Agent flow

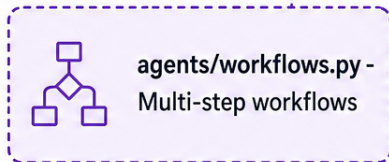
Agent flow

flowchart TD

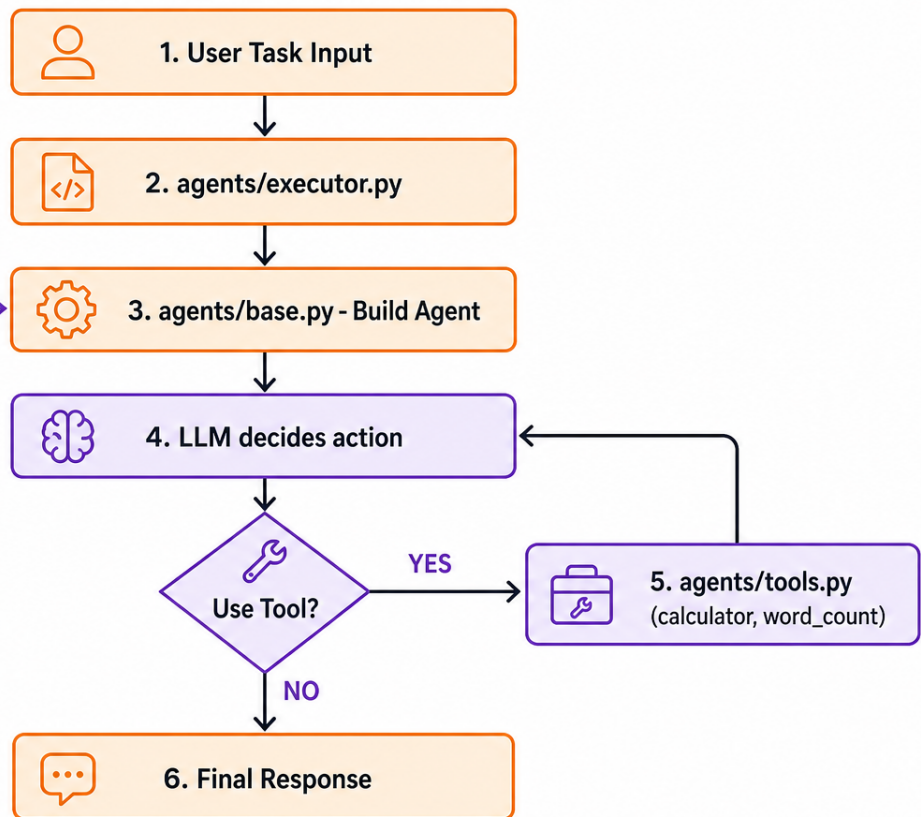
AI Agent Flow

Agent loop

1. User submits task via Agent mode



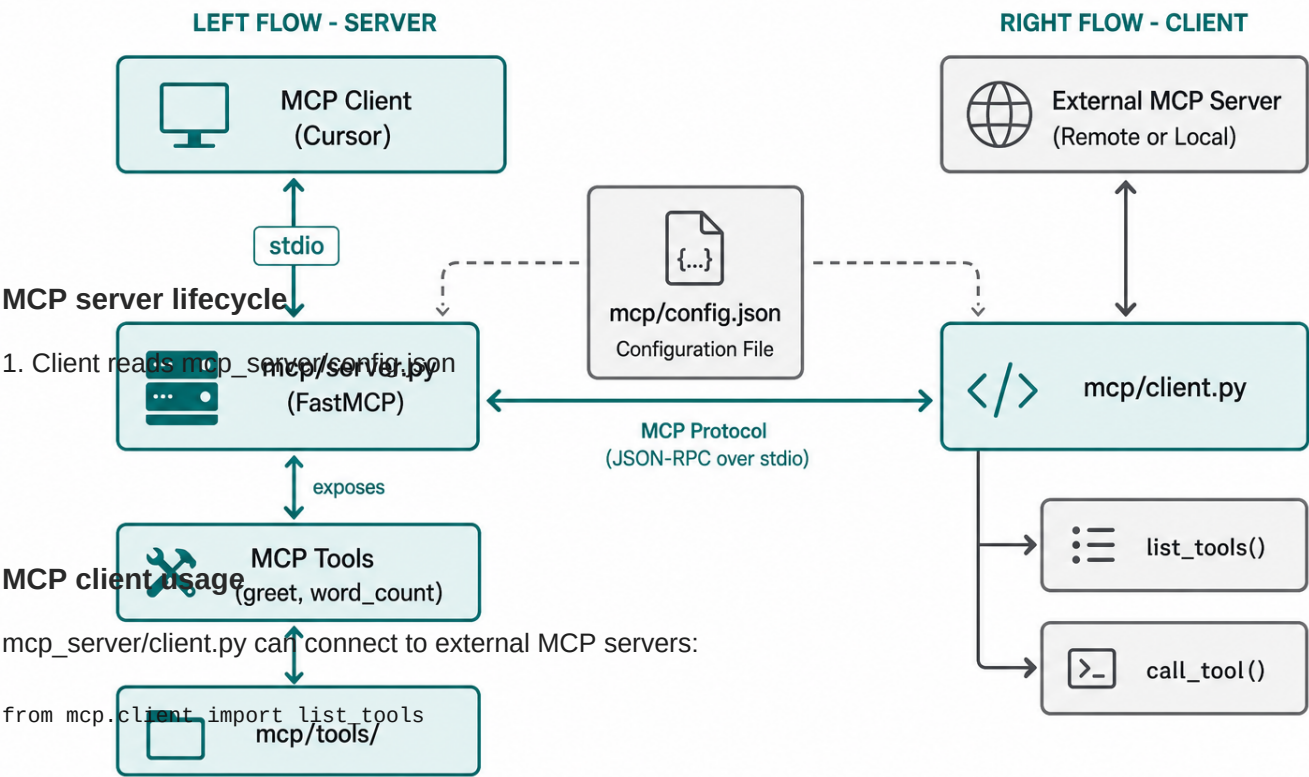
4. MCP flow



MCP flow

flowchart LR

MCP (Model Context Protocol) Flow



5. Chatbot & assistant flow

flowchart TD

6. Document ingestion decision flow

flowchart TD

7. LLM selection flow

flowchart TD

A

Models Guide

Complete guide to what models this project uses, where to get them, and all the ways to use them.

Table of contents

1. Types of models in this project

2. Wh

Types of models in this project

This project uses two kinds of AI models:

Type	Purpose	Config variable	Code module
------	---------	-----------------	-------------

??

?

LLM = the "brain" that writes answers.

Embe

Where models live in the project

Location What goes here Git tracked?

Default local LLM path:

models/llama-3-8b-instruct.Q4_K_M.gguf

Set a custom path in .env:

LLM_MODEL_PATH=models/your-model-name.gguf

Ways to use models (overview)

There are 5 ways models are used in this project:

How Entry point Models used

Additional (no LLM):

How Entry point Models used

Flow: which model runs when

flowchart TD

Option 1: OpenAI API (cloud LLM)

Best for: Agents, fast setup, no GPU, strong tool-calling.

Where to get

1. Create an account at https://platform.openai.com

2. Go

Setup

```
copy .env.example .env
```

Edit .env:

```
USE_API_LLM=true
```

OPENAI_API_KEY=

No file download needed - model runs on OpenAI servers.

Install (no extra packages)

```
uv sync --extra dev
```

langchain-openai is already included.

Run

```
uv run streamlit run app.py
```

Default model

rag/llm.py uses LangChain's ChatOpenAI with the library default model (typically gpt-4o-mini or similar depending on LangChain version).

To use a specific model, extend rag/llm.py or add to .env (future):

```
ChatOpenAI(model="gpt-4o", api_key=OPENAI_API_KEY, temperature=LLM_TEMPERATURE)
```

Cost note

API calls are billed per token. RAG with large TOP_K or long documents uses more tokens.

Option 2: Local GGUF model (on your machine)

Best for: Offline use, no API costs, full privacy.

What is GGUF?

GGUF is a compressed format for running LLMs locally via llama-cpp-python. Files end in .gguf.

Where to get GGUF models

Source URL Notes

Recommended starter models (GGUF)

Model Size (approx.) RAM needed Good for

Look for Q4_K_M quantization - good balance of speed and quality.

Download example (Hugging Face CLI)

```
# Install HF CLI if needed
```

Or download manually from the Hugging Face website -> place file in models/.

Setup

1. Place .gguf file in models/
2. Install local LLM support:

```
uv sync --extra local-llm
```

On Windows this may require Visual Studio Build Tools for llama-cpp-python. If build fails, use OpenAI API instead.

3. Configure .env:

```
USE_API_LLM=false
```

4. Run:

```
uv run streamlit run app.py
```

Quantization labels (in filename)

Label	Quality	Size	Speed
-------	---------	------	-------

Embedding models (for RAG)

Embedding models are separate from the LLM. They power document search in RAG.

Default model

```
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
```

* Size: ~80 MB (first run downloads from Hugging Face)

* Fa

Where to get / browse

Source URL

Alternative embedding models

Model Size Notes

Change in .env:

```
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
```

Important: After changing embedding model, rebuild the index:

```
uv run python ingest.py
```

How embedding model is used

```
# Step 1: ingest uses embeddings to build FAISS
```

uv run

Code path: rag/embeddings.py -> HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)

Which model for which feature?

Feature LLM required? Embedding required? Recommended LLM

Agent + local model warning

Local GGUF models via LlamaCpp often do not support tool calling well. For Agent mode, use:

USE_API_LLM=true

OPENAI

Where to download models

Quick download checklist

Model type Where Put file / setting

Hugging Face download (browser)

1. Go to a model page (e.g. TheBloke quantizations)
2. Click on the download link

Hugging Face download (command line)

uv add huggingface-hub

uv run

Verify local model file

dir models\

Ensure path in .env matches exactly:

LLM_MODEL_PATH=models\Llama-3.2-3B-Instruct-Q4_K_M.gguf

Model size & hardware guide

Your RAM Suggested GGUF LLM_N_CTX

GPU: llama-cpp-python can use GPU if built with CUDA. Default Windows install uses CPU.

Embedding models are lightweight - default model runs on CPU with ~500 MB RAM during ingest.

Configuration reference

All model settings in .env:

--- Embedding (RAG search) ---

EMBEDDING

Variable	Description
----------	-------------

Loaded in config/settings.py, used via rag/llm.py and rag/embeddings.py.

Examples: switch models

Switch to OpenAI API

```
USE_API_LLM=true
```

```
uv run streamlit run app.py
```

Switch to a different local GGUF

```
USE_API_LLM=false
```

```
uv sync --extra local-llm
```

Switch embedding model

```
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
```

```
uv run python ingest.py
```

Use models in Python directly

```
from rag.llm import get_llm
```

Troubleshooting models

Problem Solution

See also Troubleshooting.

This laptop - system profile & recommended models

Profile captured for the development machine used with this project (Windows 11, July 2026). Use this as a reference for which models run smoothly on this laptop without slowness.

Detected hardware

Component Specification

What this means on this laptop

* No NVIDIA GPU -> local LLM runs on CPU, not CUDA.

* Inte

Models that run smoothly (recommended)

Use Model Verdict

These should run without noticeable slowness for chat and RAG on this laptop.

Models that work but feel slower

Model On this laptop

Expect roughly 3-8 tokens/sec for 7B-8B models on CPU on this chip.

Avoid on this laptop

Model / setup Why

Recommended .env for this laptop (fast local)

EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2

Setup commands:

cd C:\MY_SPACE\AI_ML_GENAI

Recommended .env for this laptop (smoothest overall)

Best when you want Agent mode or zero local inference wait:

USE_API_LLM=true

Embeddings still run locally (fast on this CPU); LLM runs in the cloud.

Mode-by-mode on this laptop

App mode Smooth choice on this laptop

RAM tip for this laptop

Total RAM is 32 GB. If other apps are open and responses feel slow with a 7B-8B model:

1. Close heavy apps (browser tabs, extra IDEs).
2. Pre

Quick decision table

Goal on this laptop Use this

Related docs

- * Run, Activate & Dependencies
- * Co

Health Check & Setup Status

Last verified: July 2026 on the development laptop (Windows 11, Intel Core Ultra 5 135H, 32 GB RAM).
Use this page to confirm the project is working and to finish setup for chat, RAG answers, and agents.

Quick status

Area	Status
------	--------

What is working

Component	Details
-----------	---------

Verify yourself

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Expected: tests pass, ingest indexes chunks, MCP prints ['greet', 'word_count'].

What you still need to configure

The LLM is not ready until you choose one option below. Without this, Chatbot, AI Assistant, RAG Q&A, and Agent modes cannot generate answers.

Item	Current state	Action
------	---------------	--------

Option A - OpenAI API (fastest to test)

Best for quick testing and Agent mode.

1. Edit .env:

```
USE_API_LLM=true
```

2. Run the app:

```
uv run streamlit run app.py
```

No local model download required.

Option B - Local GGUF (this laptop)

Best for offline use. On this laptop, prefer 3B Q4_K_M for speed. See Models - this laptop profile.

1. Install local LLM support:

```
uv sync --extra local-llm
```

2. Download a model (example - fast 3B):

```
uv add huggingface-hub
```

3. Edit .env:

```
USE_API_LLM=false
```

4. Build index (if not done) and run:

```
uv run python ingest.py
```

How to run now

Always from project root:

```
cd C:\MY_SPACE\AI_ML_GENAI
```

Task Command

Open the Streamlit URL in your browser (usually `http://localhost:8501`).

After LLM is configured

Mode Expected behavior

Fixes applied during health check

These issues were found and fixed in the codebase:

Issue Fix

MCP config path: `mcp_server/config.json`

Minor warnings (non-blocking)

Warning Impact Optional fix

Full verification checklist

`cd C:\MY_SPACE\AI_ML_GENAI`

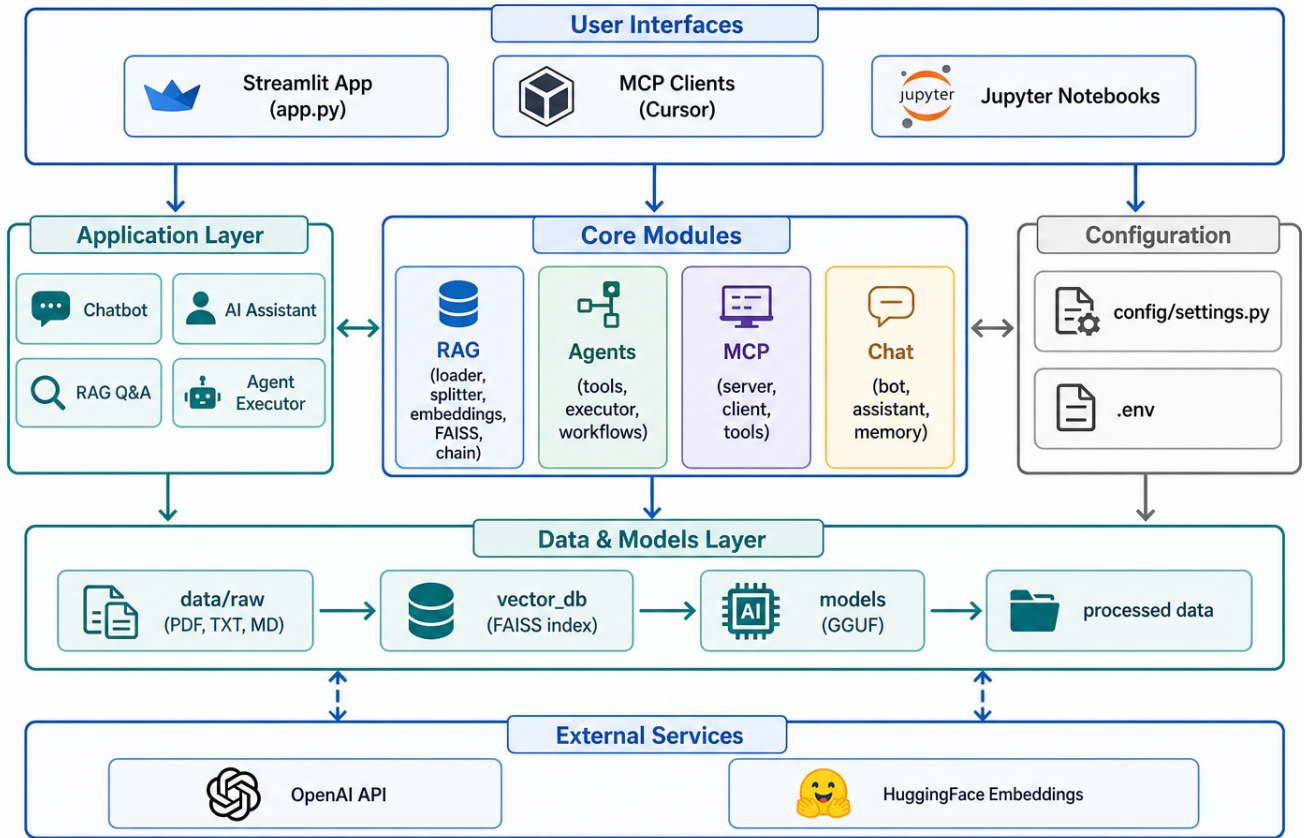
Related docs

* Run, Activate & Dependencies

* Mo

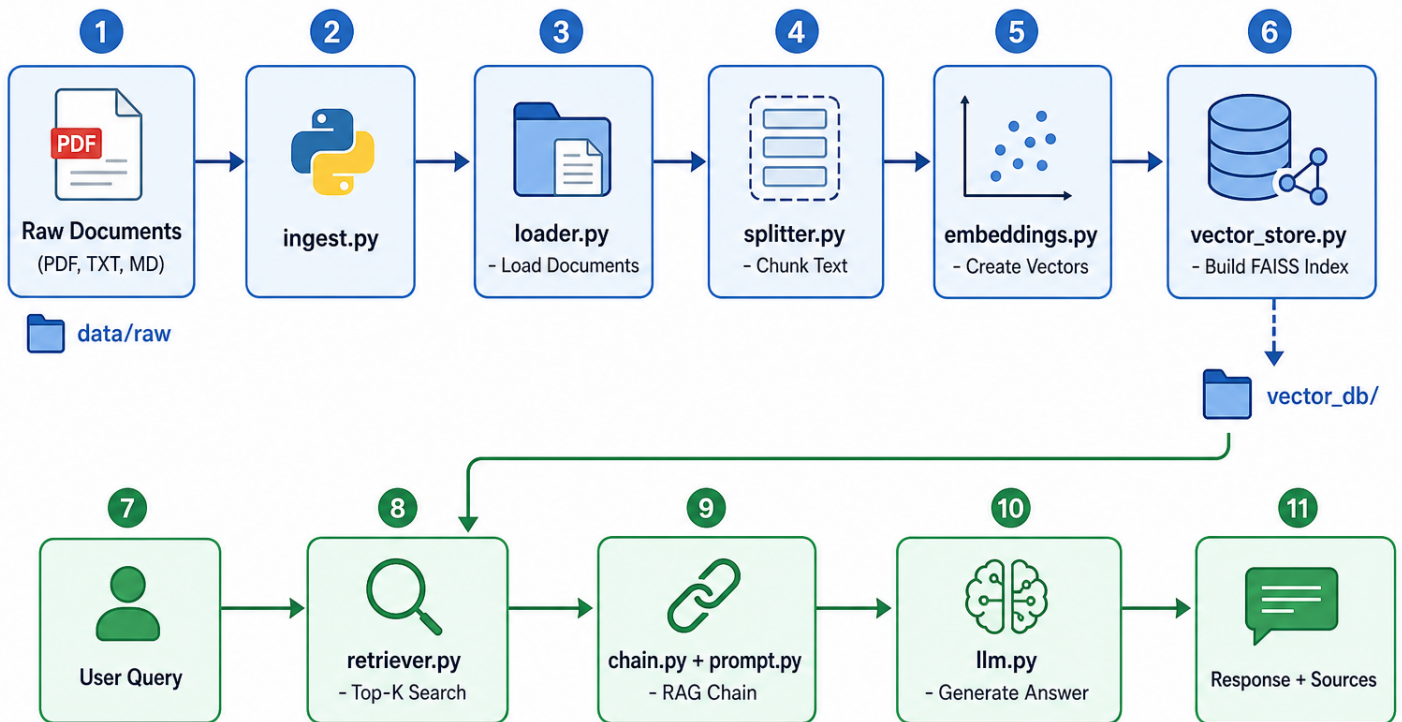
Architecture Overview

AI ML GenAI - System Architecture



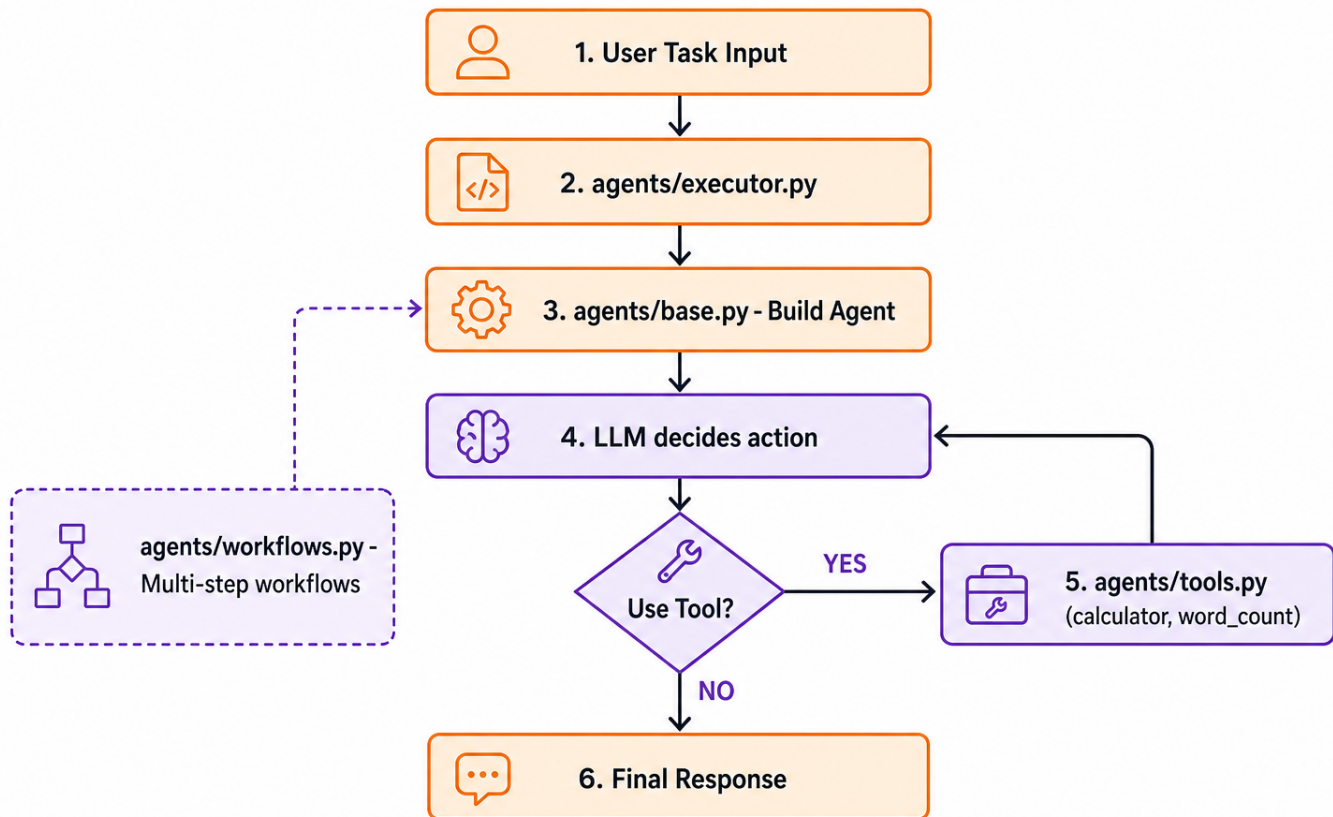
RAG Flow

RAG Pipeline Flow



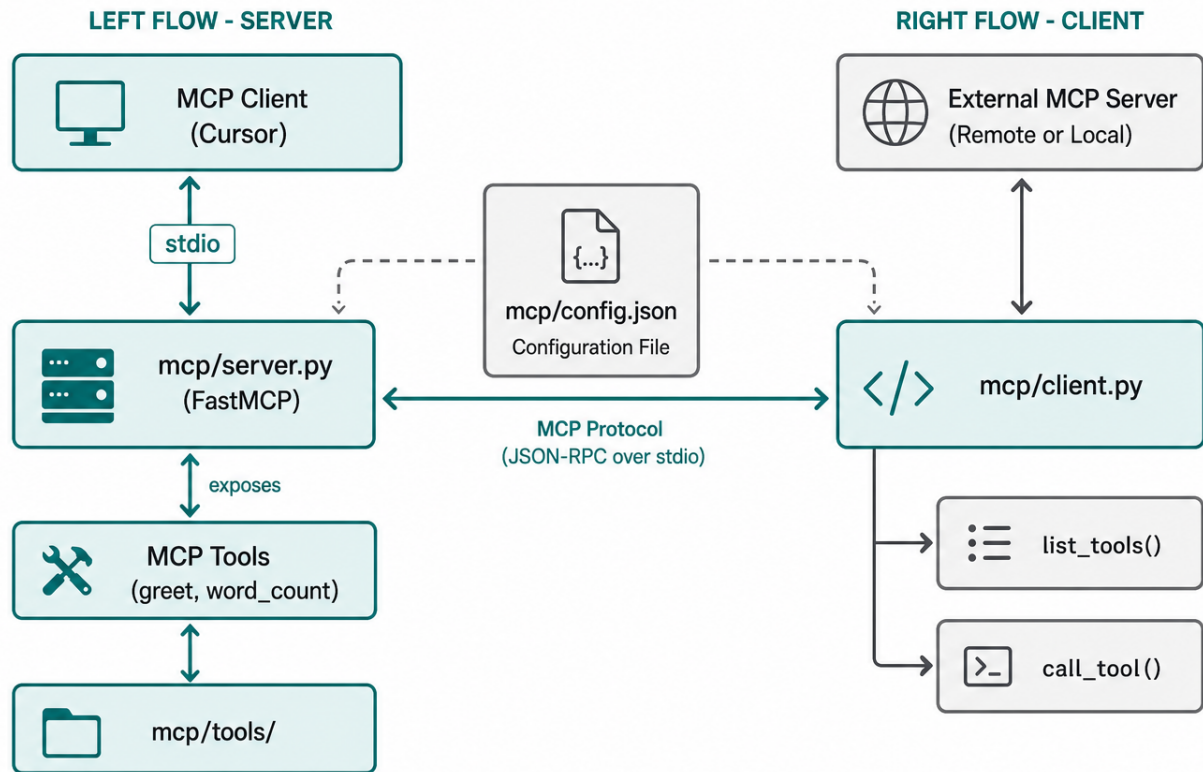
Agent Flow

AI Agent Flow



MCP Flow

MCP (Model Context Protocol) Flow



App Modes

Application Modes - app.py Streamlit UI

